

Provability Fabric: Provable Runtime Control for AI Agents

SentinelOps

August 15, 2025

Abstract

We introduce *Provability Fabric*, an architecture and artifact suite for provable control of agent runtimes. *Provability Fabric* combines *Spec-to-Proof*, which compiles ActionDSL into LEAN 4 obligations and *mechanically* synthesizes deterministic safety monitors, with a signed *Provable Agent Bundle* (PAB). The PAB is enforced by a Rust sidecar that mediates every effect over a labeled alphabet.

This work closes long-standing model–implementation gaps by (i) **mechanizing** a compact yet complete *end-to-end refinement path* (ActionDSL $\Rightarrow$ DFA tables $\Rightarrow$ interpreter run $\Rightarrow$ LEAN 4 acceptance of the same trace; see Sect. 6); (ii) generating both monitors and labelers directly from formal specifications; and (iii) shipping property-based tests with counterexample shrinking.

To address the central risk of *labeler trust*, we introduce **proof-carrying labeling**: per-field label witnesses (Merkle commitments to schema paths and taint rules) checked by the sidecar, diversity via two independent implementations (Lean-extracted and Rust), a fail-closed “no-unknown-fields” mode, and targeted negative/fuzz tests against smuggling and polyglot inputs (Sect. 13).

Within a clearly scoped TCB (adapters and OS enforcement), *Provability Fabric* provides machine-checked and runtime-enforced guarantees for authorization, declassify-aware termination-insensitive non-interference, artifact integrity, and append-only auditability. Certificates separate a global NI claim (bound to proof hashes) from a local online monitor verdict. We also document scheduler assumptions (SGEQ and a two-queue variant), define a clock model with tolerance ε , extend ActionDSL with byte- and call-budgets under streaming, and provide hardened reference adapters.

1 Introduction

LLM agents operate in *unbounded* contexts. Empirical guardrails alone cannot certify *who* may call *which* tool with *which* arguments, nor can they ensure *what* information is permitted to flow to outputs. We advocate **proof-carrying behavior**: machine-checked obligations bound to a runtime that mediates every effect through a labeled alphabet. This paper contributes end-to-end mechanization beyond sketches and introduces practical mitigations for the labeler single point of failure.

Contributions.

1. **Mechanized monitors.** We formalize a safety fragment of ActionDSL in Lean, compile it to DFA tables with *soundness and completeness proofs*, and have the sidecar interpret the generated tables directly.

2. **End-to-end micro-refinement.** We mechanize the path from ActionDSL to DFA to a simple interpreter whose trace is accepted by the same Lean semantics (Section 6), establishing the refinement theorem (Theorem 1).
3. **Proof-carrying labeling.** We generate labelers with per-field witnesses (Merkle proofs for JSON paths and taint derivations), perform dual-implementation checks, and support fail-closed exact schemas (Section 13).
4. **Labeled alphabet & declassification.** We make labels first-class in events, provide a typed grammar for Δ with well-formedness and enforcement, and include worked examples (Section 4).
5. **ActionDSL extensions.** We add byte- and call-budget rate limits with streaming semantics under clock tolerance ε and amortized $O(1)$ monitors (Section 4).
6. **Adapters & OS enforcement.** We supply frozen minimal HTTP-GET and file-read adapters with network/FS containment and explicit effect signatures (Section 12).
7. **Certificates & governance.** We bind certificates to proof/automata/labeler hashes, separate `ni_claim` from online `ni_monitor`, and constrain break-glass via M-of-N signatures, short TTLs, and auto-paging (Section 14, Section 7).
8. **Concurrency & clocks.** We use a single global event queue (SGEQ) with a linearization lemma and show a preservation lemma for a two-queue variant; the clock is monotonic with ε tolerance (Section 8).

Roadmap. Section 2 motivates the design gap. Section 3 introduces the semantic objects and notation. Section 4 presents the DSLs and policies. Sections 5 and 6 detail the mechanization, code generation, and tests. Sections 7 to 9 cover enforcement and refinement. Section 10 states system properties. The remaining sections discuss composition, adapters, labeling, certificates, replay, evaluation, scope, comparisons, admission checks, mappings, artifacts, and ethics.

2 The AI safety gap

Open-world agents lack a correctness contract. Production stacks often mix (i) prompt-time heuristics, (ii) schema checks at tool I/O, and (iii) post-hoc detectors. These reduce obvious failures but do not *state*, let alone *prove*, what is permitted under arbitrary inputs, tool chains, and environment behavior. Missing are: (1) semantics aligned with *mediation points*; (2) properties phrased as *safety* or *hyperproperties*; and (3) a refinement path tying those properties to the concrete runtime.

Observed failure modes (open-world, agentic).

- Prompt- and plan-level injection: indirect or cross-document instructions alter tool usage (*who/which/with-what*) without violating surface schemas; injections compose via retrieval and summarization.
- Tool abuse within authorization: calls are syntactically valid and “allowed,” yet argument selection, rate, or content labeling violate policy intent (e.g., silent exfiltration through permissive egress).
- Retrieval poisoning and smuggling: attacker-controlled entries and polyglot payloads (JSON-within-strings) bypass naive field checks.
- Cross-session and cross-principal flows: long-lived caches and shared adapters leak across roles or sessions when labels are not first-class.
- Covert and quasi-covert channels: length, timing, and chunking patterns carry information unless egress is deterministic up to a declared redaction.
- Replay drift and non-determinism: unpinned decoding seeds, locale/codecs variance, or unordered event handling undermine reproducibility and audit.

Root causes.

- Lack of a labeled alphabet at mediation: without labels on *events and fields*, policy cannot address what the runtime actually observes or emits.
- Hyperproperty gap: confidentiality goals are 2-safety hyperproperties and are not checkable by per-run heuristics alone [7, 17].
- Monitor-model mismatch: guardrails are informal, while enforcement runs on different structures (tokens, strings, frames); there is no proven compilation from policy to monitor.
- Unbound tool semantics: adapters are powerful but under-specified; effects (network, filesystem, time) are not modeled as part of the obligation.
- Labeler as a single point of failure: if argument/result labelers are wrong or forgeable, downstream guarantees collapse.
- Missing refinement: even a “correct” monitor can be undermined by scheduling, clock tolerance, or unmediated side paths [9].

Minimum obligations.

1. Authorization soundness: every concrete call($t, _$) is justified by a role-tool relation R under pre-state guards.
2. Policy safety via monitor acceptance: a prefix-closed language $\llbracket \varphi \rrbracket \subseteq \Sigma^*$ captured by executable monitors matching the formal semantics.
3. Declassification-aware non-interference: for $s_0 \approx_L s'_0$, permitted declassifications, and any j , the low-view verdicts agree, $\text{verdict}_L(\tau \upharpoonright j) = \text{verdict}_L(\tau' \upharpoonright j)$.
4. Labeler correctness and witness non-forgability: emitted labels equal those from schema/taint rules, and membership proofs bind to the committed path set.
5. Implementation refinement: traces under documented scheduling and clocks refine the formal semantics and are accepted by synthesized monitors.
6. Provenance integrity: proofs, automata, labelers, and artifacts are cryptographically bound to the bundle and to each emission.
7. Replay determinism up to redaction: re-execution with pinned seeds/locale/chunking reproduces low-observable behavior modulo declared redaction.

Why heuristics are insufficient.

- (a) Non-closure under composition: heuristics that “work” locally fail across pipelines; violations emerge at boundaries.
- (b) Adversarial adaptivity: once a guardrail function is learnable, distribution shifts (encoding tricks, benign prefixes) route around it.
- (c) Hyperproperty blind spot: *per-run* detectors cannot witness *relational* properties; non-interference quantifies over pairs [17].
- (d) No refinement story: without a mechanized path from specification to executable monitors, there is no end-to-end argument that “the thing you wrote” is “the thing that runs.”

Positioning. *Provability Fabric* aligns three strands, proof-carrying behavior [11, 1], runtime verification [9], and information-flow control [7, 17], to agent mediation points (calls, logs, declassify, emit-chunk/end) with a mechanized $\text{ActionDSL} \Rightarrow \text{DFA} \Rightarrow \text{interpreter} \Rightarrow \text{LEAN 4}$ path.

Assumptions and TCB. We assume complete mediation of tool IPC, filesystem writes, and egress; a monotonic clock with tolerance ε ; and SGEQ or a two-queue scheduler with FIFO-preserving

merges. Adapters and OS enforcement are in the TCB. Model internals are opaque; guarantees are stated over the labeled alphabet and monitors.

3 Definitions and notation

We fix the semantic objects, states, events, and traces, on which all properties are stated and proved. The intent is to make *observable effects* explicit and bind them to labels and witnesses so that proofs and enforcement refer to the same objects.

Labeled transition system and alphabet. Let $M = (S, \Sigma, \rightarrow)$ with $\rightarrow \subseteq S \times \Sigma \times S$ and states $s = (c, m, e)$. Every payload field carries *explicit label metadata* and, when applicable, a *label witness* identifier:

$$\begin{aligned} \Sigma \ni \text{step} \\ &| \text{call}(t, \lambda_{\text{args}}, \text{args_hash}, \text{field_commit}) \\ &| \text{ret}(t, \lambda_{\text{res}}, \text{res_hash}) \\ &| \text{log}(\lambda_{\text{msg}}, \text{msg_hash}) \\ &| \text{declassify}(\text{from_lbl}, \text{to_lbl}, \text{item_hash}) \\ &| \text{emit_chunk}(\lambda, \text{payload_hash}, i, n, \text{len}, \text{witness_id}) \\ &| \text{emit_end}. \end{aligned}$$

`field_commit` binds the set of JSON field paths via a Merkle root (high assurance) or a Bloom filter (performance), as used later in Section 13.

Labels, low equivalence, low-view. Let $(\mathcal{L}, \sqsubseteq)$ be a lattice. Atoms in c, m, e and event fields have labels via ℓ . For $L \in \mathcal{L}$, $s \approx_L s' \iff \forall a \text{ with } \ell(a) \sqsubseteq L, a \text{ agrees in } s, s'$. Write proj_L for erasing fields with labels $\not\sqsubseteq L$ (replace by erase). For an event α , $\text{view}_L(\alpha) = \text{proj}_L(\alpha)$; extend pointwise to traces.

Verdict vs. online monitor. The ideal two-run prefix verdict $\text{verdict}_L(\tau) \in \{\text{inapplicable}, \text{passed}, \text{failed}\}$ is global (not computed online). The sidecar computes a *sufficient* local monitor instead (Section 14).

Execution sets. $\text{Exec}(B, s_0)$ are the finite traces generated by the sidecar-mediated semantics under bundle B , and $\text{Exec}(B) \triangleq \bigcup_{s_0} \text{Exec}(B, s_0)$.

Clock and scheduling. A monotonic Clock with tolerance ε governs rate limits; wall-clock is audit-only. A single global event queue (SGEQ) serializes the alphabet.

Lemma 1 (Linearization). *SGEQ yields a linear extension of happens-before $\prec$ from tool IPC, stream order, and callbacks; thus every concrete run yields a linear trace in $\text{Exec}(B)$.*

Mediation. Complete mediation covers plan submission, tool IPC, filesystem writes, network egress, context mutation, declassification, and `emit_chunk/emit_end`. Excluded are raw sockets/FDs and shell escapes. Adapters are in the TCB.

4 Plan-DSL, ActionDSL, declassification, and rate limits

Plan-DSL & ActionDSL (BNF excerpt)

```
\textbf{Plan-DSL}

Plan ::= Node \{ "," Node \}* \\\
Node ::= call(name,\underline{lbl\_map},args\_hash,\underline{field\_commit}) \\\
\phantom{Node ::=}\; \; \; \log(\underline{lbl},msg\_hash)
\; \; \; declassify(label\_h \to label\_l,item\_hash)
\; \; \; emit(plan\_id)
\; \; \; retrieve(source,receipt\_hash) \\\
Args ::= JSON\ (schema-validated;\ hash=sha256(JCS(JSON));\ field\_commit over names/
        paths)

\smallskip
\textbf{ActionDSL (safety fragment)}

ActionCode ::= allow Role ":" Tool "when" always  $\Phi$ 
\ \ \ forbid Event "when"  $\Phi$ 
\ \ \ rate\_limit\_calls Tool "within"  $\Delta$  "bound"
\ \ \ rate\_limit\_bytes Egress "within"  $\Delta$  "bound" \\\
 $\Phi ::=$  atomic
\ \ \  $\Phi$ 
\ \ \  $\Phi$ 
\ \ \ always  $\Phi$ 
\ \ \ within  $\Delta$  at\_most \\\
Event ::= call(name, args\_hash)
\ \ \ log(msg\_hash)
\ \ \ declassify(l\_h \to l\_l, item\_hash)
\ \ \ emit(plan\_id)
```

Design intent. Plan-DSL pre-authorizes effects (calls, logs, declassifications, emits) and binds them to labels and committed field sets. ActionDSL states *prefix-closed* obligations that can be compiled into monitors and checked online.

Sliding-window semantics. At index j , `rate_limit_calls` holds when $|\{i \leq j \mid t_j - t_i < \Delta + \epsilon, \text{ matches}\}| \leq \kappa$. For `rate_limit_bytes`, the sum of emitted byte lengths over matching events in the window is $\leq \beta$. Implementations use dequeues; memory $O(\kappa)$ or $O(\beta/\text{chunk})$ with amortized $O(1)$ operations.

Declassification policy Δ . Rules range over principals (`role`, `tool`, or `any`); justifications can be combined conjunctively. Well-formedness requires labels in $\mathcal{L}$, acyclic flows monotone in $\sqsubseteq$, and typed principals. Each concrete `declassify` must match a rule; the sidecar enforces this before forwarding.

Plan completeness and smuggling resistance. Every mediated call/log/declassify/emit must be pre-authorized by a matching plan node and compatible label metadata. `field_commit` binds the exact path set (Merkle mode) or detects anomalies (Bloom mode). `-no-unknown-fields` rejects extraneous fields.

5 Spec-to-Proof interface and monitors

Generated Lean 4 interface.

proofs/Spec.lean (key excerpts; all definitions, no axioms)

```

1 namespace PF
2
3 structure Role      := mk :: (name : String)
4 structure Effects   := mk :: (tags : List String)
5 structure Context   := mk :: (bytes : ByteArray)
6 structure Memory    := mk :: (bytes : ByteArray)
7 structure Env       := mk :: (bytes : ByteArray)
8 structure State     := (c : Context) (m : Memory) (e : Env)
9
10 inductive Event
11 | step
12 | call      (tool : String) (lbl_map : String) (args_hash : String) (field_commit :
      String)
13 | ret       (tool : String) (lbl_map : String) (res_hash : String)
14 | log       (lbl : String) (msg_hash : String)
15 | declassify (from_lbl to_lbl : String) (item_hash : String)
16 | emit_chunk (lbl : String) (payload_hash : String) (idx tot : Nat) (len : Nat) (
      witness_id : String)
17 | emit_end
18 deriving DecidableEq
19
20 structure Step := (s : State) (evt : Event) (s' : State)
21 abbrev Trace := List Step
22 structure Tool := (name : String) (eff : Effects)
23
24 structure Semantics :=
25   (Allowed      : Role → Tool → Prop)
26   (SidecarWitness : Type)
27   (Checked      : Step → SidecarWitness → Prop)
28   (Invariants   : List (Trace → Prop))
29   (NonInterf    : Trace → Prop)
30
31 inductive Mediated (sem : Semantics) : Trace → Prop
32 | nil : Mediated []
33 | cons (stp : Step) (w : sem.SidecarWitness) (h : sem.Checked stp w)
34   (τ : Trace) (ih : Mediated τ) : Mediated (stp :: τ)
35
36 def Conj (Invs : List (Trace → Prop)) (τ : Trace) : Prop :=
37   ∀ inv ∈ Invs, inv τ
38
39 def BundleSafeType (sem : Semantics) :=
40   (τ : Trace) → Mediated sem τ → (Conj sem.Invariants τ ∧ sem.NonInterf τ)
41
42 end PF

```

Monitor synthesis and complexity. Let $\llbracket \varphi \rrbracket \subseteq \Sigma^*$ be the set of finite prefixes that satisfy φ (prefix-closed safety, including windows under ε). For each φ we synthesize a DFA $\mathcal{A}_\varphi$ with

$L(\mathcal{A}_\varphi) = \llbracket \varphi \rrbracket$. The bundle monitor $\mathcal{M}_B \triangleq \prod_\varphi \mathcal{A}_\varphi$ uses deny-wins. Each $\mathcal{A}_\varphi$ is $O(|\varphi|)$; window checks use dequeues with the complexities above.

6 Mechanization, code generation, and tests

Automata synthesis in Lean. We formalize ActionDSL’s safety fragment, define trace semantics, implement a compiler to DFA tables, and *prove* that each generated DFA recognizes $\llbracket \varphi \rrbracket$. **End-to-end micro-refinement.** We formalize a small interpreter that consults DFA tables and emits a mediated trace, and prove that its run is accepted by the same Lean semantics.

proofs/Spec.lean (key excerpts; all definitions, no axioms)

```

1 namespace PF
2
3 /-- Deterministic finite automaton tables emitted in the bundle. -/
4 structure DFA := ( : Type) [DecidableEq ]
5   (start : ) (acc : → Bool)
6   ( : → Event → )
7
8 /-- Product DFA for a bundle (elides codegen details). -/
9 structure DFAM := ( : Type) [DecidableEq ]
10   (start : ) (acc : → Bool)
11   ( : → Event → )
12
13 /-- Small-step interpreter state carries the DFAM control. -/
14 structure IState := ( : DFAM.) (st : State)
15
16 /-- One interpreter step: consult , check guards, advance , emit Step. -/
17 def interp_step (M : DFAM) (sem : Semantics)
18   (is : IState) (evt : Event) (st' : State)
19   (guard_ok : True) : IState × Step :=
20   let ' := M.is.evt
21   let step := Step.mk is.st evt st'
22   (IState.mk ' st', step)
23
24 /-- A finite run produces a list of Steps (a Trace). -/
25 def interp_run (M : DFAM) (sem : Semantics)
26   (init : IState) (es : List (Event × State)) : IState × Trace := ...
27
28 /-- Acceptance of a Trace by the DFAM control path. -/
29 def accepts (M : DFAM) : Trace → Prop := ...
30
31 /-- Main micro-refinement theorem. -/
32 theorem micro_refine
33   (M : DFAM) (sem : Semantics)
34   (init : IState) (es : List (Event × State)) :
35   let (_, τ) := interp_run M sem init es
36   Mediated sem τ ∧ accepts M τ :=
37 by
38   -- proof by induction on es; each step uses guard_ok and
39   ...
40

```

Theorem 1 (End-to-end micro-refinement). *For the DFAM tables generated from a set of ActionDSL clauses and the small interpreter above, every finite run yields a trace τ such that $\text{Mediated}(\text{sem}, \tau)$ and the DFAM accepts τ .*

Mechanized proof idea. By induction on the event list: each step consults δ , advances the DFAM control, attaches a witness for `sem.Checked`, and preserves mediation. Acceptance follows the DFAM control path. Because the DFAM tables are those proved correct for the clauses, the chain connects $\text{ActionDSL} \rightarrow \text{DFA} \rightarrow \text{interpreter run} \rightarrow \text{Lean acceptance}$. $\square$

Code generation. The bundle contains DFA tables (states, initial, accepting, transitions) and their hash; the sidecar interprets these tables directly.

Property-based tests. CI generates random traces (including adversarial chunking) and checks equivalence between the Rust interpreter and a Lean reference evaluator; shrinkers minimize counterexamples.

7 Cryptography, provenance, and governance

Bundles are deterministic tarballs with DSSE/Cosign envelopes. Rekor transparency is checked against a pinned STH (keys via TUF). For *availability fallback*, we mirror logs; if unavailable, a grace window is permitted under break-glass with M-of-N operator signatures, short TTL, auto-page, and a recorded post-mortem. Revocation freshness is $\leq 24\text{h}$; refresh occurs every 30 min. The monotonic clock governs enforcement; wall-clock is audit-only.

8 Runtime enforcement, clocks, and scheduler

On admission, the sidecar verifies signatures, transparency (or governed fallback), SBOM policy, proof obligations, and DFA/labeler hashes. Decoding parameters and seeds are pinned.

Mediation hooks. We intercept plan submission, tool IPC, filesystem writes, network egress, context mutation, declassification, and `emit_chunk/emit_end`. Model sockets are disallowed; all egress flows through the sidecar.

Guards. The system enforces $(u, t) \in R$, pre-state guards, adapter witnesses (schema-checked args, ABAC receipts for retrievals, rate-limit tokens, idempotency/signed receipts). Violations fail closed.

Clock & tolerance. We use `CLOCK_MONOTONIC` or equivalent; ε and wrap are documented. Rate limits widen windows to $\Delta + \varepsilon$.

Scheduler. SGEQ is the default. A two-queue variant (calls vs egress) preserves safety under FIFO-preserving merges when cross-queue events commute with respect to guards/monitors.

9 Implementation refinement

Theorem 2. *Fix B and a sidecar build implementing SGEQ (or the two-queue with preservation preconditions), the clock API with tolerance ε , complete mediation, and DFAM interpretation using the generated tables. Then every concrete run on B produces τ with $\text{Mediated}(\text{sem}, \tau)$, where `sem` is generated from B .*

Outline. Combine Lemma 1 with Theorem 1 and the fact that the sidecar enforces guards/monitors using the same DFAM tables and semantics extracted from B . The argument proceeds by induction on prefixes; deny-wins prevents any violating step. $\square$

10 Properties as theorems

Authorization soundness. If admission succeeds for B and $\text{Mediated}(\text{sem}, \tau)$, then every $\text{call}(t, _)$ occurs under some role u with $(u, t) \in R$.

Safety via monitors. If $\bigwedge_i \varphi_i$ type-checks and $\text{Mediated}(\text{sem}, \tau)$, then every prefix of τ is accepted by $\mathcal{M}_B$.

Termination-insensitive NI with declassification. Under mediation, typed Δ , and the NI obligation, for any τ' with $s_0 \approx_L s'_0$ and permitted declassifications, $\text{verdict}_L(\tau \upharpoonright j) = \text{verdict}_L(\tau' \upharpoonright j)$ for all j ; in particular, when $\text{verdict}_L = \text{passed}$, view_L agrees.

11 Composition contracts and micro-protocols

Adapters μ are restricted to bounded scripts with tagged success/error that preserve label/order and authorization; product semantics include error-tagged steps. A worked HTTP-GET adapter expands to $\text{DNS} \rightarrow \text{TCP} \rightarrow \text{GET} \rightarrow \text{read}(\text{N})$ with fixed DNS, no redirects, enforced **Content-Length**, and explicit codecs.

12 Adapters, effects, and OS enforcement

HTTP-GET (frozen). Fixed DNS, URL allow-list, no redirects, required/enforced **Content-Length**, explicit codecs only, no HTTP/2 push, no chunked transfer unless declared, with CA/ALPN pinned.

File-read. Read-only bind mounts, symlinks forbidden, path normalization, and recording of (inode, device, hash); egress uses deterministic chunking.

Effect signatures. $\text{Eff} ::= \text{net}(\text{allow_urls}) \mid \text{fs}(\text{paths}, \text{mode}) \mid \text{clock}$. Namespaces and **seccomp** are enforced. Embedding/vector DB adapters are out of scope unless a conservative taint rule is adopted.

13 Proof-carrying labeling: design and mitigation

Generator from schema + taint. The labeler is *generated* from schema and taint rules; no handwritten mapping is used. For each JSON path p , the generator emits a label $L(p)$ and a *witness*:

- **Path commitment.** A Merkle tree over canonicalized paths with $\text{field_commit} = \text{root}$. For each present path, a Merkle proof is included in a **LabelCert**; the sidecar verifies membership.
- **Taint derivation.** A derivation tree showing how $L(p)$ arises from schema and taint rules (joins, overrides). The sidecar checks a compact encoding of this derivation (bounded-depth fragment).

Dual implementations (diversity). The sidecar evaluates labels with two independent implementations, (i) a Lean-extracted reference and (ii) the Rust generator, on the *same* canonicalized JSON; any mismatch fails closed.

Exact schemas for risky tools. In **-no-unknown-fields** mode, any path without a witness is rejected. In performance mode, a Bloom filter raises a low-FPR alarm for unexpected fields; high-assurance deployments use Merkle commitments only.

Negative tests and polyglots. CI injects random permutations/insertions, deeply nested objects, and polyglot payloads (JSON-within-strings) to ensure labels and plan authorization cannot be bypassed.

Labeler soundness properties. We prove in Lean for the schema fragment:

- **Projection totality/canonicity:** JCS re-serialization preserves `args_hash` and the label map.
- **Monotonicity:** In exact mode, adding new fields is rejected; in projection mode, existing field labels do not change.
- **Witness non-forgability:** Any accepted path has a valid Merkle proof and a well-typed taint derivation with respect to the schema.

14 Egress filtering and certificates

Detectors (regex/FSM, SimHash) are *defense-in-depth only*. Streaming filters run per `emit_chunk` and again on the concatenation at `emit_end`. Certificates explicitly separate global claims from local checks and bind all generators.

Local NI monitor and bridge to global NI

Runtime predicate (decidable; online). For a level L , we define a decidable, prefix-closed predicate $\text{MonNI}_L(\tau)$ computed online by the sidecar from label metadata and declassification records:

- If $\text{obs}_L(\alpha_j)$ is false, accept trivially for index j .
- If $\text{obs}_L(\alpha_j)$ is true, the sidecar requires witnesses that the set of contributing labels S_j for α_j satisfies $S_j \subseteq \text{Allow}_\Delta(L, \tau \upharpoonright j)$, where Allow_Δ is the smallest set containing L and closed under Δ -permitted flows that have actually occurred on the prefix (witnessed declassify events). The witness must validate (i) path membership in the committed field set and (ii) the taint derivation that produced each label in S_j (cf. §13). Any mismatch $\Rightarrow$ reject.

We write $\text{MonNI}_L(\tau \upharpoonright j) = \text{accept}$ when the above holds for every index $\leq j$; *inapplicable* if no obs_L occurs; *reject* otherwise.

Unwinding obligations (static; in-bundle). The bundle carries a mechanized proof of the following *local unwinding* obligations for the labeled alphabet:

- **Output consistency (OC).** For any $s \approx_L s'$ and step $s \xrightarrow{\alpha} s_1$ with $\text{obs}_L(\alpha)$, if α 's contributing labels S are $\subseteq \text{Allow}_\Delta(L, \tau)$, then there exists s' -step $s' \xrightarrow{\alpha'} s'_1$ with $\text{view}_L(\alpha) = \text{view}_L(\alpha')$ and $s_1 \approx_L s'_1$.
- **Step compatibility (SC).** For non- obs_L events, low projections of states and events are preserved: if $s \approx_L s'$ and $s \xrightarrow{\alpha} s_1$ with $\neg \text{obs}_L(\alpha)$, then there exists $s' \xrightarrow{\alpha'} s'_1$ with $\text{view}_L(\alpha) = \text{view}_L(\alpha')$ and $s_1 \approx_L s'_1$.
- **Declassify admissibility (DA).** Every declassify step matches a typed Δ rule (cf. §4) and augments Allow_Δ monotonically.

Lemma 2 (Local monitor soundness w.r.t. unwinding). *If the unwinding obligations (OC, SC, DA) type-check for bundle B and $\text{MonNI}_L(\tau \upharpoonright j) = \text{accept}$, then for any $s'_0 \approx_L s_0$ and any $\tau' \in \text{Exec}(B, s'_0)$ respecting Δ we have $\text{view}_L(\tau \upharpoonright j) = \text{view}_L(\tau' \upharpoonright j)$.*

Theorem 3 (Local-to-global NI bridge). *Assume Mediated(sem, τ), the unwinding obligations hold for B , and $\text{MonNI}_L(\tau \upharpoonright j) = \text{accept}$ for all j . Then $\text{verdict}_L(\tau \upharpoonright j) = \text{passed}$ for all j and $\text{NI}(\tau)$ holds termination-insensitively w.r.t. Δ .*

Proof sketch. Induction on j . At each step, apply Lemma 2 to $\tau \upharpoonright j$ versus any $\tau' \upharpoonright j$ from a low-equivalent initial state with permitted declassifications; equality of low views follows. Termination-insensitivity follows by prefix reasoning. $\square$

Per-emission certificate.

pab.json (PAB manifest)

```

1 {
2   "bundle_id":"sha256:...",
3   "session_id":"sess...",
4   "seq":1287,
5   "policy_hash":"sha256:...", // canonical ActionDSL hash
6   "proof_hash":"sha256:...", // NI obligation hash
7   "automata_hash":"sha256:...", // DFA tables hash
8   "labeler_hash":"sha256:...", // labeler generator hash
9   "label_witness_hash":"sha256:...", // Merkle root / witness set hash
10  "lean_commit":"...", // Lean code commit id
11  "sidecar_build":"...", // sidecar build id
12  "ni_claim":{"proof_hash":"sha256:...", "level":"L"},
13  "ni_monitor":"inapplicable|accept|reject|error", // local sufficient-condition
    verdict (\MonNI_L)
14  "detectors":{"pii_regex_set":"v2025.08", "simhash_seed":"0xC0FFEE00", "near_dup_thresh":0.85},
15  "blocked_spans":[[123,147],[201,216]],
16  "redaction_summary":{"pii":2, "secret":0, "near_dup":1},
17  "attestation_ref":"quote:...", // optional TEE
18  "sig":"dsse:..."
19 }
```

Bridge guarantee. When `ni_monitor=accept` for all prefixes and the bundle’s unwinding proof checks, Theorem 3 yields $\text{verdict}_L = \text{passed}$ for the same prefixes.

Prefix verdict. Here, `ni_monitor` is the local sufficient-condition verdict over the prefix; `ni_claim` binds the global NI proof by hash.

15 Replay determinism

Replays use the exact `bundle_id`, fixtures, decoding parameters, and a deterministic redaction ρ . Determinism holds *up to* ρ preserving low fields; divergences raise `E_REPLAY_DRIFT`. The sandbox pins `LANG/LC_*`, timezone, newline translation, float locale/rounding, tokenizer/chunker versions; egress is deterministic via fixed `-chunk-bytes` and `-flush-ms`.

16 Evaluation and artifact checks

Testbed

Node	CPU	RAM	Storage	OS	Sidecar	Monitors / states (avg)
pf-bench-01	8-core x86-64 @ 3.2GHz	32GB	NVMe	Ubuntu 24.04	1.3.2	14 / 9.7

We use 20 concurrent sessions with local FS logging and light event rates.

Per-event timings (monitor + hash-chain append)

Sample (ms)	p50	p95	p99	Notes
0.089, 0.098, 0.122	0.098	0.122*	0.122*	*small- N estimate

Other artifact checks

- **Fixture-backed replays:** 3/3 passes; 0 drift events.
- **Labeler negatives:** permutations/insertions/polyglot inputs rejected as intended; exact-match mode blocks unknown fields.
- **Monitor interpreter equivalence:** property tests pass; shrinking exercised on seeded faults.

Throughput note. SGEQ is the default secure mode. We report p95 throughput ceilings and evaluate a two-queue variant in Part II.

17 Comparison to alternatives

	Proof-backed	Online enforcement	Declassification	Provenance
PF (our work)	✓	✓	✓	✓
Heuristic guardrails	—	✓	—	—
Runtime monitors (no proofs)	—	✓	~	—
Policy engines (OPA) [†]	—	✓	~	~

[†]“~” denotes feasibility with extensions.

18 Reproducibility and artifacts

Repository: <https://github.com/SentinelOps-CI/provability-fabric>.

Proof build: `cd proofs && lake build`.

Automata export: `lake exe export-dfa --out=artifact/dfa.json`.

Dual labeler build: `lake exe export-labeler && cargo build -p labeler`.

Rust tests: `cargo test -p sidecar --features prop`.

Deterministic egress: `--chunk-bytes=4096 --flush-ms=40 --locale=C.UTF-8 --tz=UTC`.

Evidence: `evidence/egress_certs/*.json`, `evidence/safety_case/*.zip`,

`replay_results.json`, `evidence/postmortem-template.md`.

19 Assumptions, limitations, and ethics

Detectors are defense-in-depth, not confidentiality proofs. Confidentiality excludes covert timing/length channels unless deterministic egress is enabled. Guarantees degrade if mediation is bypassed or adapters misdeclare effects. Labeler diversity and witnesses reduce risk but do not eliminate it; high-assurance deployments should enable `-no-unknown-fields`. Audit logs/fixtures may include sensitive metadata; configure retention and sealing. Break-glass remains tightly governed and audited.

References

- [1] A. W. Appel and E. W. Felten. Proof-carrying authentication. In *Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS '99)*, pages 52–62, Singapore, 1999. Association for Computing Machinery.
- [2] L. Bauer, S. Garriss, and M. K. Reiter. Distributed proving in access-control systems. In *Proceedings of the 2005 IEEE Symposium on Security and Privacy (S&P '05)*, pages 81–95, Oakland, CA, USA, 2005. IEEE Computer Society.
- [3] M. Davis and M. Dürst. Unicode standard annex #15: Unicode normalization forms. <https://www.unicode.org/reports/tr15/>, 2024. Version current as of 2024; Accessed 2025-08-16.
- [4] L. de Moura and S. Ullrich. The lean 4 theorem prover and programming language. In A. Platzer and G. Sutcliffe, editors, *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [5] s. Developers. cosign: Container signing, verification, and storage in an oci registry. <https://github.com/sigstore/cosign>, 2025. Accessed 2025-08-16.
- [6] s. Developers. rekor: Software supply chain transparency log. <https://github.com/sigstore/rekor>, 2025. Accessed 2025-08-16.
- [7] J. A. Goguen and J. Meseguer. Security policies and security models. In *Proceedings of the 1982 IEEE Symposium on Security and Privacy (S&P '82)*, pages 11–20, Oakland, CA, USA, 1982. IEEE Computer Society.
- [8] in-toto Project. Dead simple signing envelope (dsse) specification. <https://github.com/secure-systems-lab/dsse>, 2023. Accessed 2025-08-16.
- [9] M. Leucker and C. Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- [10] Linux Foundation SPDX Workgroup. The software package data exchange (spdx) specification, version 2.3. <https://spdx.github.io/spdx-spec/v2.3/>, 2022. Accessed 2025-08-16.
- [11] G. C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*, pages 106–119, Paris, France, 1997. Association for Computing Machinery.
- [12] G. C. Necula and P. Lee. The design and implementation of a certifying compiler. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation (PLDI '98)*, pages 333–344, Montreal, Canada, 1998. Association for Computing Machinery.
- [13] Z. Newman, J. S. Meyers, and S. Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. Association for Computing Machinery, 2022.
- [14] OWASP Foundation. Cyclonedx bill of materials (bom) standard, ECMA-424, 1st edition. <https://www.ecma-international.org/publications-and-standards/standards/ecma-424/>, 2024. Accessed 2025-08-16.
- [15] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS 2002)*, pages 55–74, Copenhagen, Denmark, 2002. IEEE Computer Society.
- [16] A. Rundgren. Json canonicalization scheme (jcs). RFC 8785, RFC Editor, June 2020.
- [17] A. Sabelfeld and A. C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.
- [18] Sigstore Project. Sigstore documentation: Cosign, fulcio, and rekor. <https://docs.sigstore.dev/cosign/signing/overview/>, 2025. Accessed 2025-08-16.

- [19] SLSA Community. Slsa specification, version 1.0. <https://slsa.dev/spec/v1.0/>, 2023. Accessed 2025-08-16.
- [20] The Update Framework (TUF). The update framework (tuf) specification, version 1.0. <https://theupdateframework.github.io/specification/>, 2024. Accessed 2025-08-16.
- [21] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappelletti. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 2019)*, pages 1393–1410, Santa Clara, CA, USA, 2019. USENIX Association.